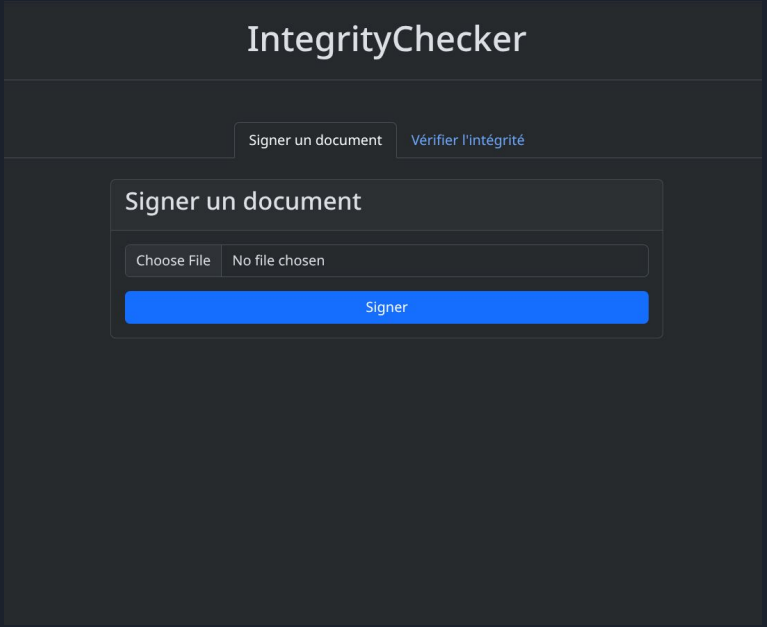
Integrity-Checker

Objectif : Attester de l'intégrité et de l'authenticité d'un document

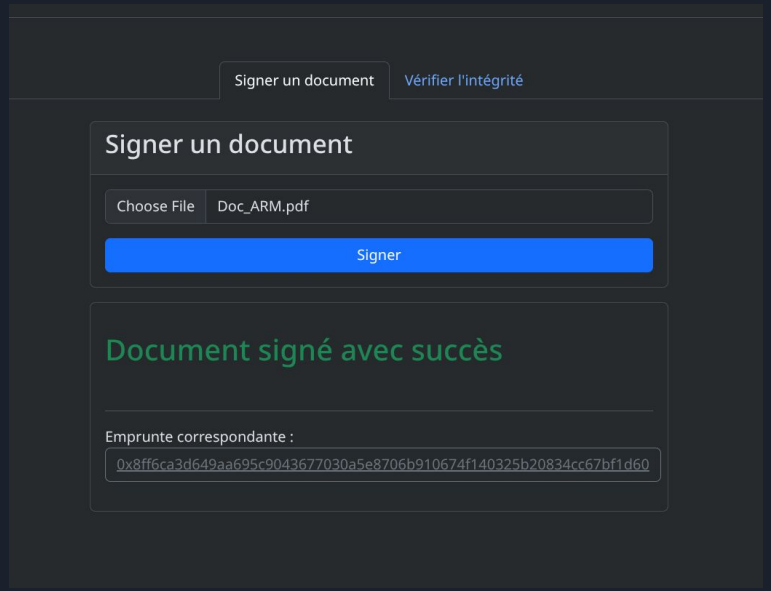
Deux fonctionnalités :

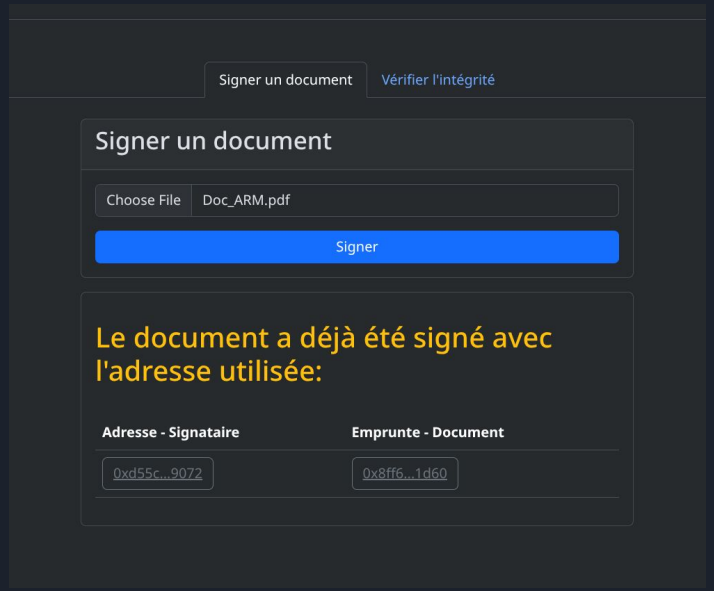
- Signer un document :
  - Inscrit la signature du document dans la Blockchain
  - Délivre un certificat d'authenticité
- Vérifier l'intégrité d'un document
  - Vérifier la présence du document dans la Blockchain et affichage du/des signataires
  - Utiliser le certificat avec le document pour attester son authenticité

Interface simple



Signature





- Signature rapide après validation de la transaction dans MetaMask

- Une seule signature par document et adresse : **immutabilité**

Certificat

CERTIFICAT DE SIGNATURE

Ce document atteste de l'authenticité et de l'intégrité du fichier signé.

Nom du fichier :
Doc_ARM.pdf

Date de signature :
Thu, 19 Mar 2026 08:40:28 GMT

Adresse du signataire :
0xd55c13d7a1adb81c6884cd0322123ebf0ac09072

Adresse du Smart-Contract :
0x8ff6ca3d649aa695c9043677030a5e8706b910674f140325b20834cc67bf1d60

Emprunte SHA256 du document :
0x8ff6ca3d649aa695c9043677030a5e8706b910674f140325b20834cc67bf1d60

[Important]! Ce certificat n'est pas lui même signé en raison de l'absence de chaine de certification.
Assurez vous d'en avoir pris possession de façon fiable.

Ce certificat a été généré automatiquement et atteste de l'intégrité du fichier signé.

- ➔ Certificat au format PDF:
  - ◆ Facilement lisible
  - ◆ Concis
- ➔ Contient les informations clés en métadonnées:
  - ◆ Signataire
  - ◆ Adresse du contrat
  - ◆ Hash du document
- ➔ Généré dans Node avec pdf-lib

Validation

Signer un document | Vérifier l'intégrité

Vérifier l'intégrité d'un document

Choose File Doc_ARM.pdf

☐ J'ai un certificat

Signatures

#	Adresse - Signataire	Date - Signature
1	0xd55c...9072	Thu Mar 19 2026 09:40:28 GMT+0100 (Central European Standard Time)
2	0xfD6e...bd1e	Thu Mar 19 2026 09:57:48 GMT+0100 (Central European Standard Time)

Signer un document | Vérifier l'intégrité

Vérifier l'intégrité d'un document

Choose File Doc_ARM.pdf

☑ J'ai un certificat

Choose File certificat_Doc_ARM.pdf (1).pdf

Certificat valide

#	Adresse - Signataire	Date - Signature
1	0xfD6e...bd1e	Thu Mar 19 2026 09:57:48 GMT+0100 (Central European Standard Time)

```
pragma solidity ^0.8.0;

contract IntegrityChecker{

    int128 public constant OK=0;
    int128 public constant ALREADY_SIGNED=8;

    event addSignatureResult(address signataire, bytes32 hash, int128 code);

    struct Signature{
        uint timestamp;
        address signataire;
    }

    // mappings
    mapping (
        bytes32 => Signature[]
    ) signatures;

    mapping (
        bytes32 => mapping(
            address => bool
        ) ) ont_signe;


    function getSignatures(bytes32 hash) external view returns (Signature[] memory){
        return signatures[hash];
    }

    function hasAddrAlreadySigned(bytes32 hash)public  view returns(bool){
        return ont_signe[hash][msg.sender];
    }

    function addSignature(
        bytes32 hash
    ) external {
        if(hasAddrAlreadySigned(hash)){
            emit addSignatureResult(msg.sender, hash, ALREADY_SIGNED);
        }

        signatures[hash].push(
            Signature({
                timestamp: block.timestamp,
                signataire: msg.sender
            })
        );
        ont_signe[hash][msg.sender] = true;
        emit addSignatureResult(msg.sender, hash, OK);
    }
}
```

Smart-Contract

- Mapping **signatures** qui contient toutes les signatures
  - Utilise l'empreinte comme clé
- Mapping **ont_signe** pour déterminer si le sender a déjà signé le document
  - Accès en O(1)
- Getters appelés avec call() par web3js
  - **getSignatures** : renvoie le tableau des signatures pour un hash
  - **hasAddrAlreadySigned** : recherche dans ont_signe : retourne false par défaut
- **AddSignature**
  - Ajoute la signature
  - Émet un évènement avec un code de retour

UI

- ➔ Html
- ➔ Bootstrap
- ➔ Vanilla js

```html
<!DOCTYPE html>
<html lang="fr" data-bs-theme="dark">
<head>
    <title>IntegrityChecker</title>
    <link href="css/bootstrap.min.css" iel="stylesheet">
    <link href="css/main.css" rel="stylesheet">
    <script src="js/web3.min.js"></script>
    <script src="js/pdf-lib.min.js"></script>
    <script src="js/bootstrap.bundle.min.js"></script>
    <script src="js/helpers.js"></script>
    <script src="js/ajax.js"></script>
    <script src="js/blockchain.js"></script>
    <script type="module" src="js/index.js" defer></script>

    <meta charset="utf-8">
</head>

<body class="d-flex flex-column min-vh-100">

<header class="container-fluid py-3 border-bottom">
    <div class="row align-items-start">

        <div class="col-md-3" id="header-left"></div>


        <div class="col-md-6 text-center" id="header-center">
            <h1>IntegrityChecker</h1>
        </div>

        <div class="col-md-3 d-flex justify-content-end position-fixed end-0 top-0" id="header-right">
            <div class="card p-2 w-100" style="max-width: 350px;">
                <div class="card-body p-2 text-center">

                    <div id="provider"></div>
                    <div id="chainId"></div>
                    <div id="latestBlock"></div>


                    <h6 class="mt-2"><button type="button" class="btn btn-primary-dark" data-bs-toggle="collapse" data-bs-targ
                    <div class="collapse show" id="collabse-block-infos">
                        <hr class="my-2">

                        <small>Numéro</small>
                        <div id="latestBlock-number" class="ps-2 bg-dark bg-gradient rounded"></div>

                        <small>Nonce</small>
                        <div id="latestBlock-nonce" class="ps-2 bg-dark bg-gradient rounded"></div>

                        <small>Hash</small>
                        <div id="latestBlock-hash" class="ps-2 bg-dark bg-gradient rounded"></div>

                    </div>
                    <div id="connectedAccount" class="mt-2"></div>
                </div>
            </div>
        </div>

    </div>
</header>

<main class="container flex-grow-1 my-5">

    <ul class="nav nav-tabs justify-content-center" id="nav-tab" role="tablist">
        <li class="nav-item">
            <button class="nav-link active" id="nav-sign-tab"
                data-bs-toggle="tab" data-bs-target="#nav-sign"
                type="button" role="tab">
                Signer un document
            </button>
        </li>
        <li class="nav-item">
            <button class="nav-link" id="nav-verify-tab"
                data-bs-toggle="tab" data-bs-target="#nav-verify"
                type="button" role="tab">
                Vérifier l'intégrité
            </button>
        </li>
    </ul>
```

```javascript
const BLOCKCHAIN = new BlockChain();


const CODES = {
    OK : 0,
    ALREADY_SIGNED : 8
}

BLOCKCHAIN.setOnNewBlockCallback( (block)=>{
    console.log("[*] NEW BLOCK : ",block);
    document.getElementById("latestBlock-number").innerText = block.number;
    document.getElementById("latestBlock-nonce").innerText = block.nonce;
    document.getElementById("latestBlock-hash").innerHTML = `<button type="button" class="btn btn-outline-secondary clickable" data-copy
});
await BLOCKCHAIN.setContract("IntegrityChecker");

BLOCKCHAIN.onNewBlock(await BLOCKCHAIN.getBlock( await BLOCKCHAIN.getBlockNumber()));

BLOCKCHAIN.subscribeToBlocksChange();


/**
 *
 * @returns {Object}
 */
function getVerifyState() {
    return {
        hasCert: document.getElementById("i-have-cert").checked,
        file: document.getElementById("file-verify-input").files[0],
        cert: document.getElementById("cert-verify-input").files[0]
    };
}

/**
 *
 * @param {Event} e
 * @returns
 */
async function handleVerification(e) {
    e.preventDefault();

    document.getElementById('verify-result').innerHTML = "";
    if( !BLOCKCHAIN.accounts){
        console.log("initAccount()");
        await BLOCKCHAIN.initAccount();
    }

    const state = getVerifyState();

    if (!state.file) {
        return showVerifyError("Veuillez sélectionner un fichier");
    }

    const hash = await getHashOfDoc(state.file);
    const signatures = await BLOCKCHAIN.verifySignature(hash);

    if (state.hasCert) {
        return handleWithCertificate(state, hash, signatures);
    } else {
        return handleWithoutCertificate(signatures);
    }
}

/**
 *
 * @param {Array<Object>} signatures
 * @returns
 */
function handleWithoutCertificate(signatures) {
    if (signatures.length === 0) {
        return showVerifyError("Document non reconnu");
    }

    return showSignaturesTable(signatures);
}

/**
 *
 * @param {Object} state
 * @param {String} hash
```

Interactions avec MetaMask & Node

```c
function shortAddr(addr, firstchars=6) {
    return addr.slice(0, firstchars) + "..." + addr.slice(-4);
}

/**
 *
 * @param {File} FILE
 */
function getHashOfDoc(FILE){
    console.log(FILE);

    return new Promise((resolve, reject) => {

        const reader = new FileReader();

        reader.onload = async () => {
            try{
            const hashBuffer = await window.crypto.subtle.digest("SHA-256", reader.result);
            const hashHex = [...new Uint8Array(hashBuffer)]
                .map(x => x.toString(16).padStart(2, '0'))
                .join('');

            resolve("0x" + hashHex.toLowerCase());
            }
            catch(e){
                reject(e);
            }
        }
        reader.error = reject;
        reader.readAsArrayBuffer(FILE);
    });

}



function readFileAsArrayBuffer(file) {
    return new Promise((resolve, reject) => {
        const reader = new FileReader();
        reader.onload = () => resolve(reader.result);
        reader.onerror = (err) => reject(err);
        reader.readAsArrayBuffer(file);
    });
}


/**
 *
 * @param {Array<Object>} signatures
 */
function showSignaturesTable(signatures){
    const verify_result = document.getElementById("verify-result");
    // on met le tableau des signatures quand même
    verify_result.innerHTML = "<h2>Signatures</h2>"
    const table = document.createElement("table")
    table.classList.add("table","table-dark");
    table.innerHTML += `<thead>
    <tr>
    <th scope="col">#</th>
    <th scope="col">Adresse - Signataire</th>
    <th scope="col">Date - Signature</th>
    </tr>
    </thead>`;

    const tbody = document.createElement("tbody");

    for(let i=0; i<signatures.length; i++){
        let tr = document.createElement("tr");
        tr.innerHTML += `
        <tr>
            <th scope="row">${i+1}</th>
            <td><button type="button" class="btn btn-outline-secondary clickable" data-copy="${s
            <td>${new Date(Number(signatures[i].timestamp)*1000)}</td>
        </tr>
        `;
        tbody.appendChild(tr);
    }
    table.appendChild(tbody);
    verify_result.appendChild(table);
```

```c
class BlockChain {
    constructor() {
        this.web3 = new Web3(window.ethereum);
        this.contractInfos = null;
        this.accounts = null;
        this.contract = null;
        this.contractName = null;
        this.fetcher = new Fetcher();
        this.blockSubscription = null;
        console.log(this);
        this.onNewBlock = null;
    }
    isMetaMask() {
        return window.ethereum.isMetaMask;
    }

    setOnNewBlockCallback(onNewBlock){
        this.onNewBlock = onNewBlock;
    }

    async getChainId() {
        return await this.web3.eth.getChainId();
    }
    async getBlockNumber() {
        return await this.web3.eth.getBlockNumber();
    }
    async initAccount() {
        this.accounts =await window.ethereum.request({ method: "eth_requestAccounts" });
    }
    async getBlock(blockNumber){
        return await this.web3.eth.getBlock(blockNumber);
    }


    async subscribeToBlocksChange() {
        this.blockSubscription = await this.web3.eth.subscribe('newBlockHeaders');
        this.blockSubscription.on('data', block => {
            if (this.onNewBlock){
                this.onNewBlock(block);
            }
        });
    }
    async setContract(contractName) {
        console.log("Blockain state : ", this);
        this.contractName = contractName;
        this.contractInfos = await this.fetcher.fetchContract(contractName);
        console.log("[*] Contract fetched", this.contractInfos);
        this.contract = new this.web3.eth.Contract(this.contractInfos.abi, this.contractInfos.address);
    }
    async addSignature(hash) {
        console.log("Blockain state : ", this);
        return this.contract.methods.addSignature(hash).send({
            from: this.accounts[0],
        });
    }


    async verifySignature(hash){
        console.log("Blockain state : ", this);
        return await this.contract.methods.getSignatures(hash).call({
            from: this.accounts[0],
        });
    }

    async hasAddrAlreadySigned(hash){
        console.log("Blockain state : ", this);
        return await this.contract.methods.hasAddrAlreadySigned(hash).call({
            from: this.accounts[0],
        });
    }


}
```

```c
function  Fetcher(){
}


Fetcher.prototype.fetchPdfAjax = async (file, signerAddress, documentHash)=>{
    const response = await fetch('/certificate', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
    },
    body: JSON.stringify({
        fileName: file.name,
        signerAddress: signerAddress,
        documentHash: documentHash
    })
    });
    const blob = await response.blob();
    const url = window.URL.createObjectURL(blob);

    const a = document.createElement('a');
    a.href = url;
    a.download = `certificat_${file.name}.pdf`;
    a.click();

    window.open(url, '_blank').focus();

}



 Fetcher.prototype.fetchContract = async (contractName) => {
    try{
        const response = await fetch(`/contracts/${contractName}.json`, {
            headers : {
                "Content-Type" : "application/json"
            }
        });

        const json = await response.json();
        return json;
    }
    catch(e){
        console.log(e);
        return {};
    }
}
```

Création du certificat sur Node

```javascript
import { Router } from 'express';
const router = Router();

import { generateCertificate } from '../services/certificate.js';
import { BlockchainWrapper } from '../services/blockchain.js';


router.post('/certificate', async (req, res) => {
    try {
        const { fileName, signerAddress, documentHash } = req.body;

        const blockchain = new BlockchainWrapper("IntegrityChecker");

        // on utilise le compte de l'utilisateur
        blockchain.accounts = [signerAddress];

        const signature = await blockchain.getSignature(documentHash, signerAddress);
        if( !signature){
            throw new Error("Impossible de trouver la signature correspondante");
        }

        const signatureTimestamp = signature.timestamp;
        const signDate = new Date(Number(signatureTimestamp)*1000).toUTCString();
        const contractAddress = blockchain.contractJson.address.toLowerCase();

        const pdfBytes = await generateCertificate({
            fileName,
            signDate,
            signerAddress,
            documentHash,
            contractAddress
        });

        res.setHeader('Content-Type', 'application/pdf');
        res.setHeader('Content-Disposition', 'attachment; filename="certificate.pdf"');
        res.send(Buffer.from(pdfBytes));
    } catch (error) {
        console.error(error);
        res.status(500).send('Erreur lors de la génération du certificat' +  error.message );
    }
});

export default router;
```

- ➔ Endpoint /certificate créé avec le framework Express
- ➔ Génération avec pdf-lib
- ➔ Compilation, déploiement du contrat et de l’ABI automatiquement
- ➔ Intégration de l’adresse du client dans le certificat

Merci de votre attention